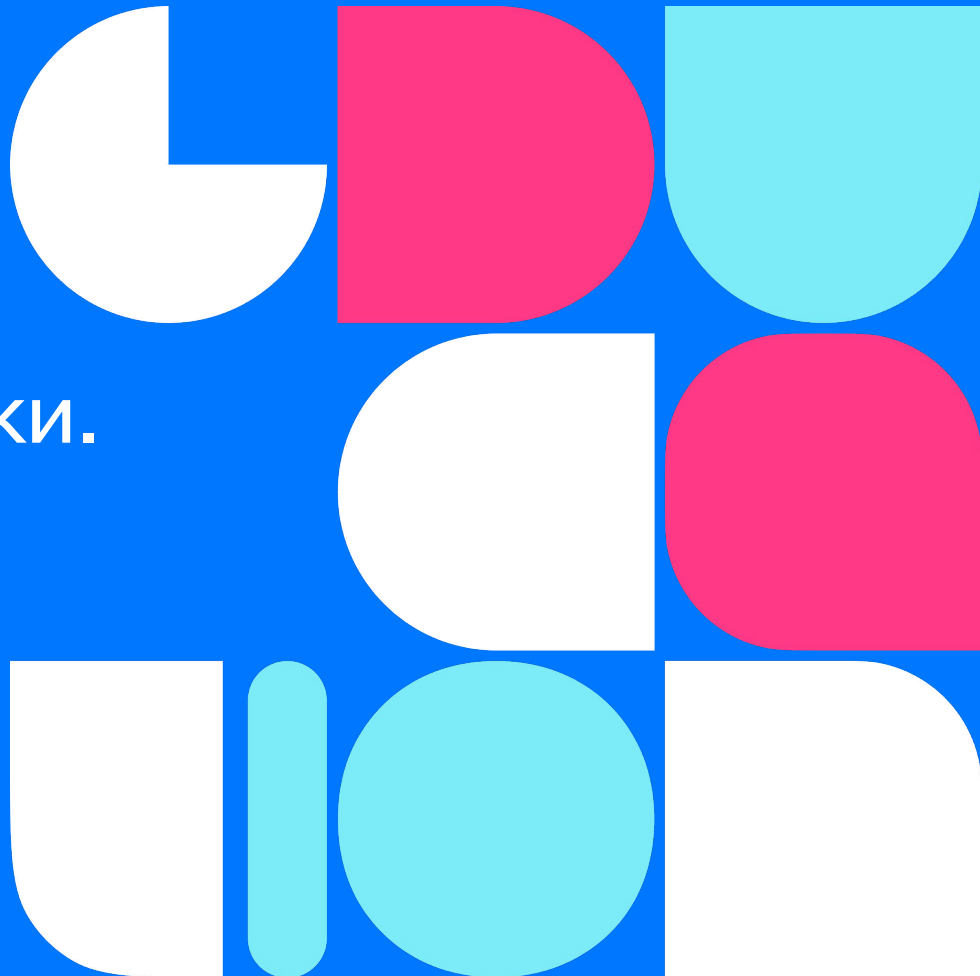
education

Очереди. Деки и стеки. Связные списки

Семинар



Константин Жуков

Старший инженер по тестированию,
Вконтакте



education

Цель занятия:

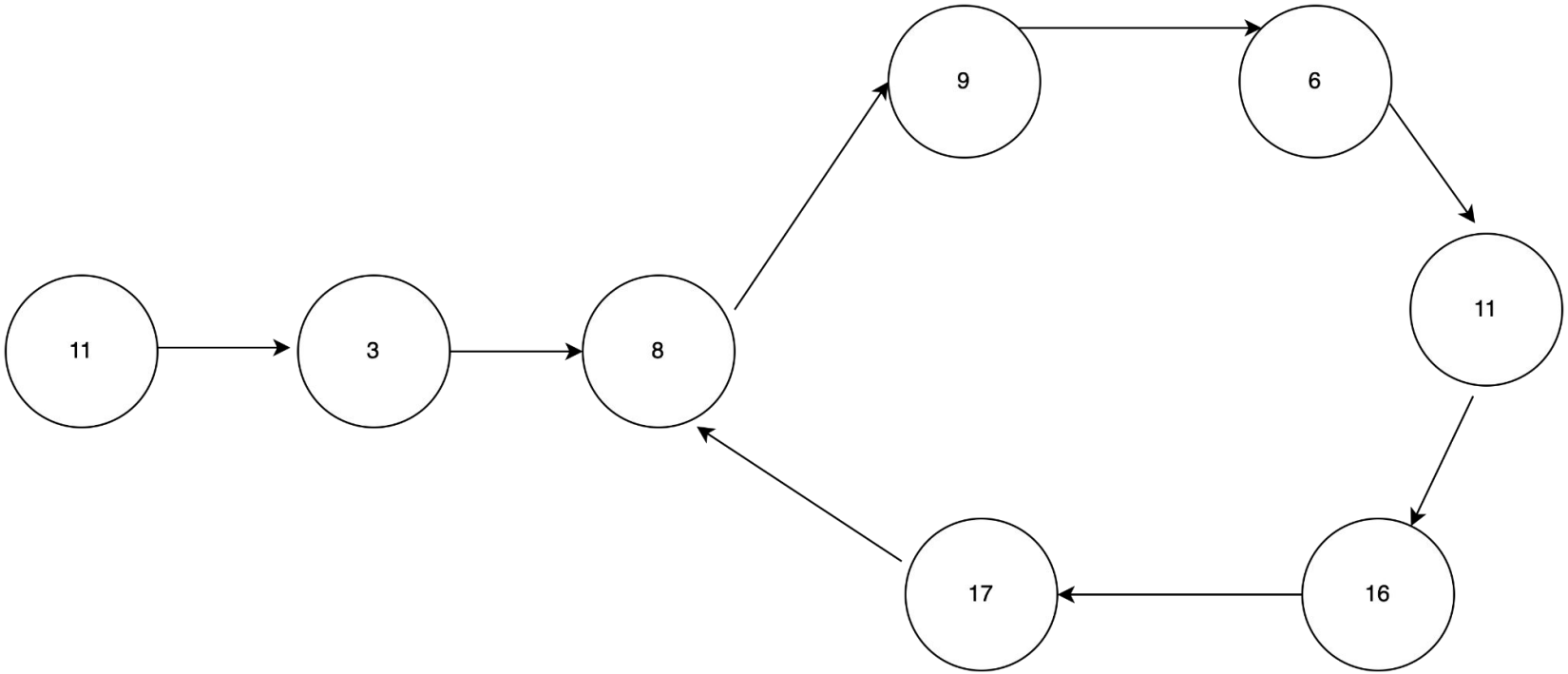
Научиться применять очереди, деки, стеки и связные списки для решения задач.

Проверить, является ли список циклическим

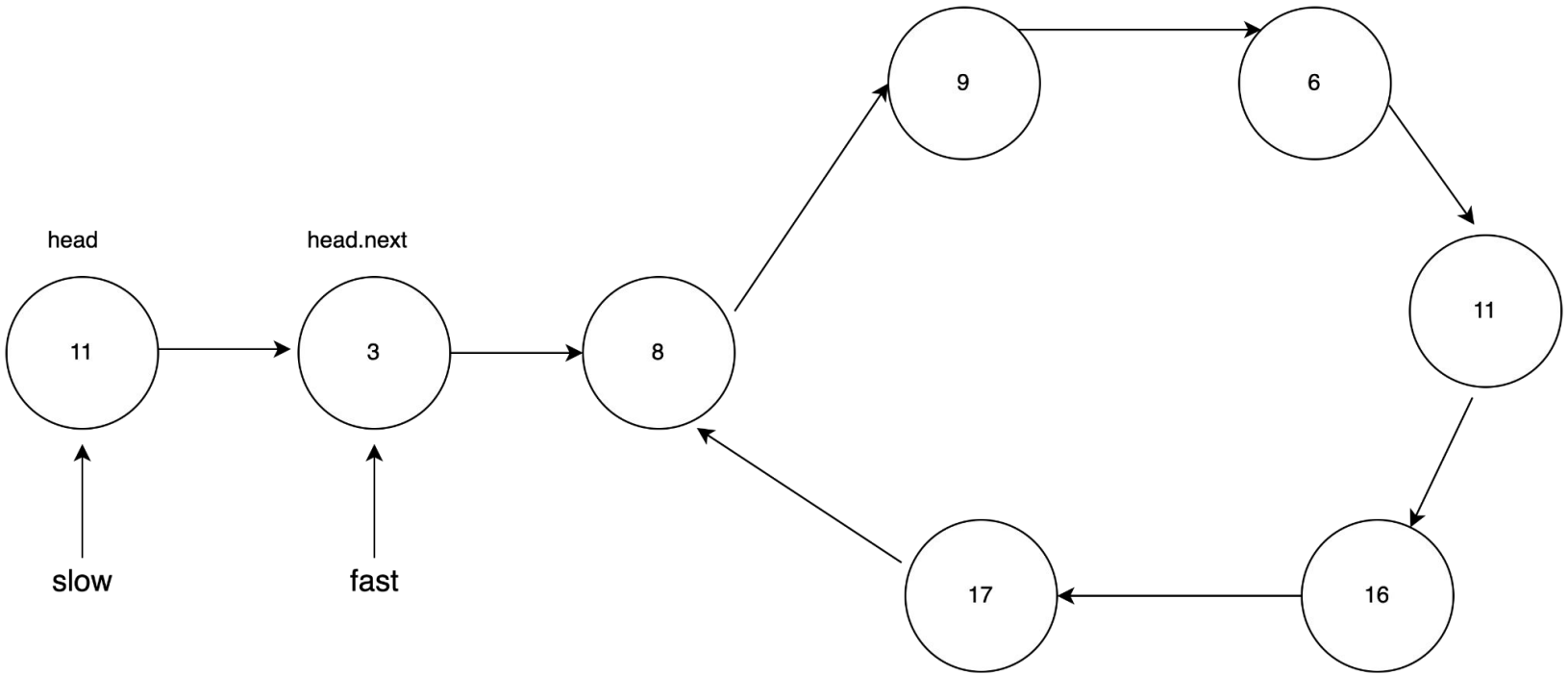
Дан односвязный список.
Необходимо проверить, является ли этот список циклическим.

Циклическим (кольцевым) списком называется список у которого последний узел ссылается на один из предыдущих узлов.

Циклический список

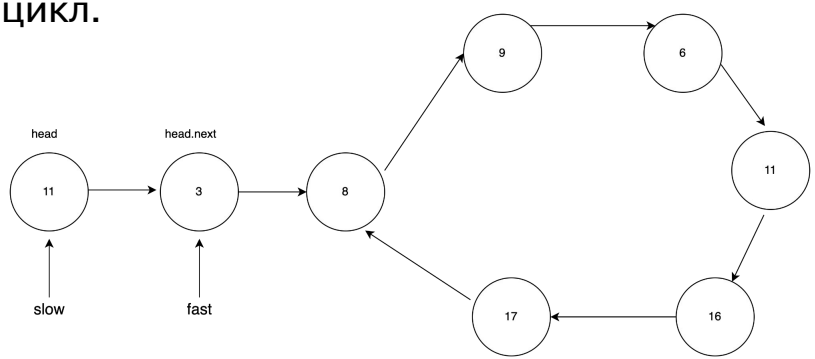


Циклический список



Является ли список циклическим

Проходимся в цикле по нашему списку. Если быстрый указатель дошел до null, значит список не циклический. Если медленный указатель в какой-то момент стал равен быстрому, значит мы нашли цикл.




```
function hasCycle(head) {
    if head == null || head.next == null {
        return false
    }

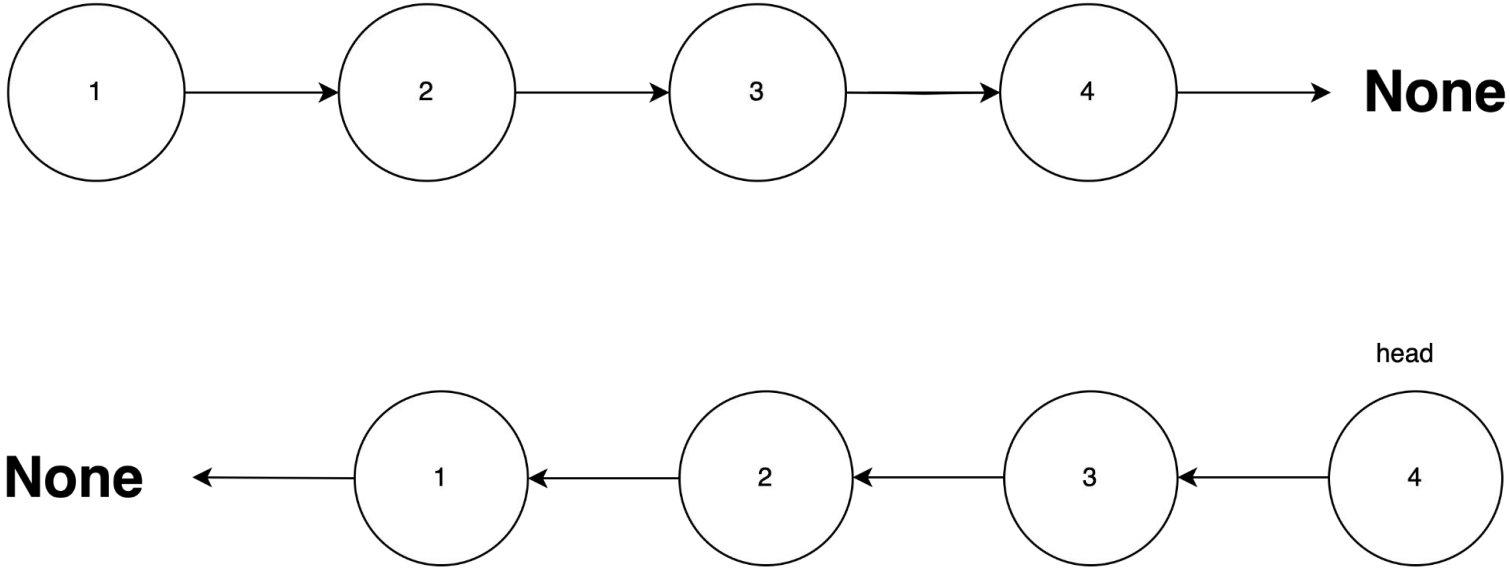
    slow = head
    fast = head.next

    while slow != fast {
        if fast == null || fast.next == null {
            return false
        }
        slow = slow.next
        fast = fast.next.next
    }
    return true
}
```

Развернуть односвязный список

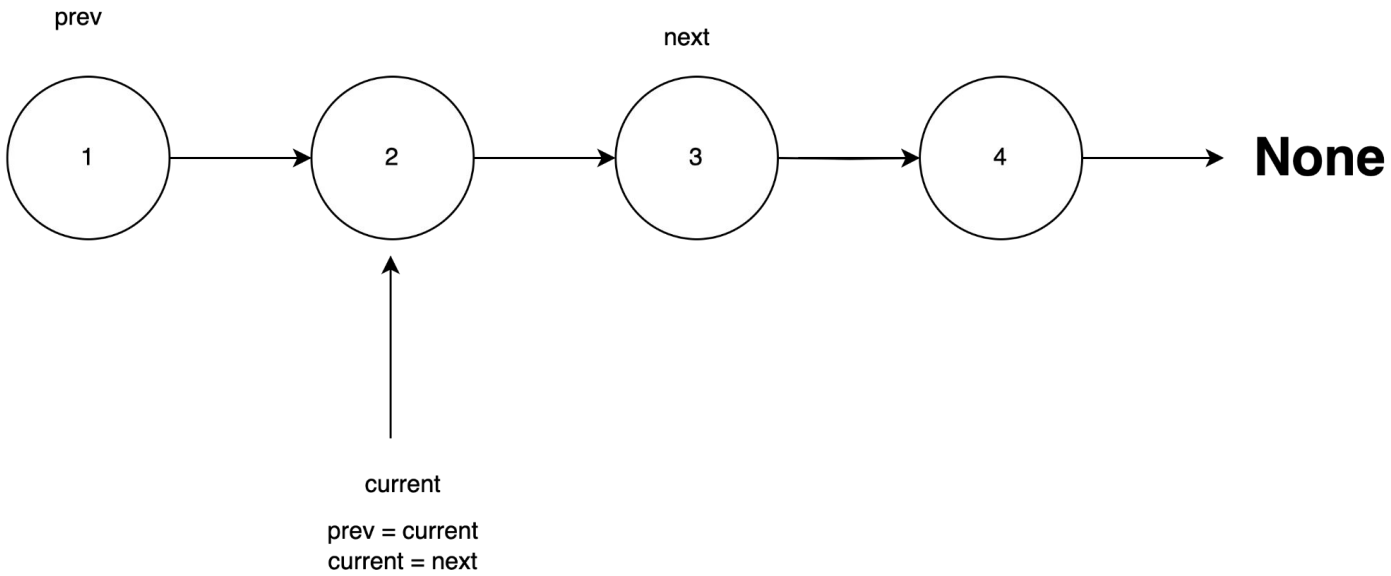
Необходимо написать функцию, которая принимает на вход односвязный список и разворачивает его.

Reverse linked list

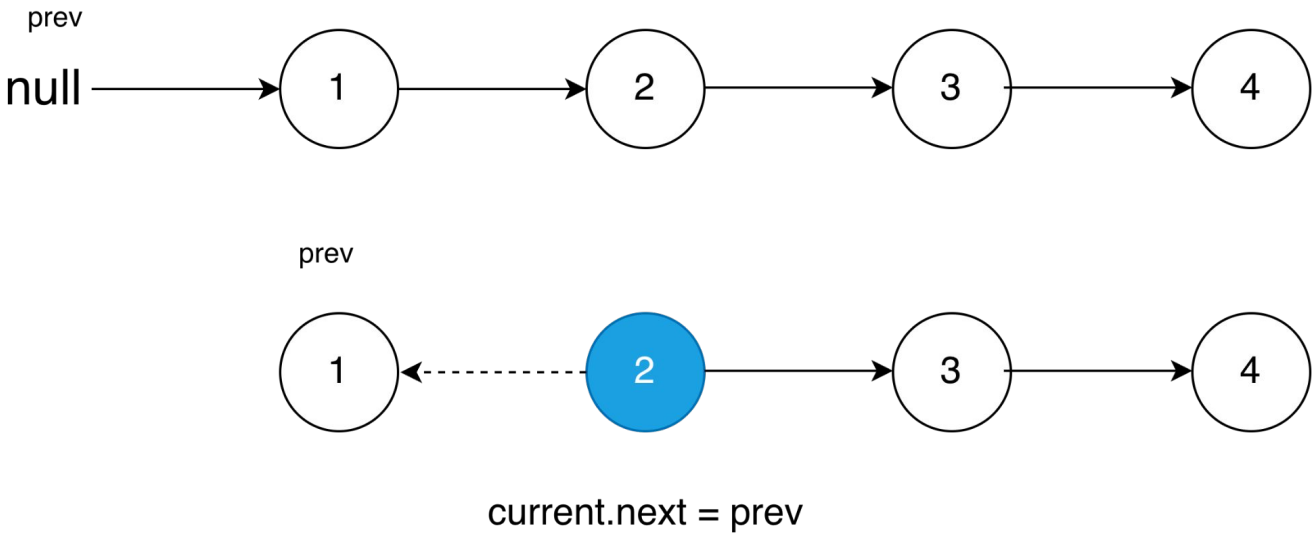


reverseLinkedList

- Проходим последовательно в цикле по всему списку
- На каждой итерации меняем указатели у узлов

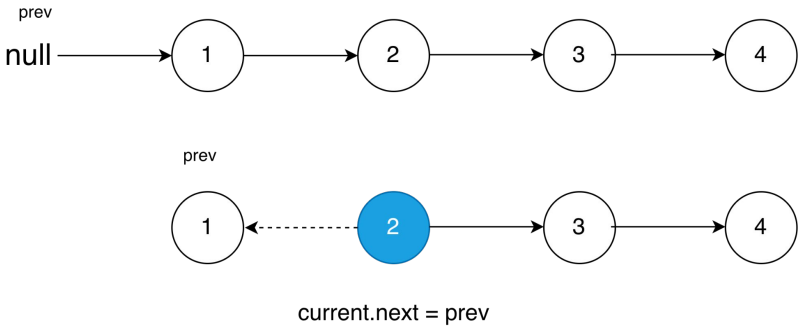


Reverse linked list



reverseLinkedList

- Проходим последовательно в цикле по всему списку
- На каждой итерации меняем указатели у узлов




```
function reverseLinkedList(head) {
    prev = null
    current = head

    while (current !== null) {
        next = current.next
        current.next = prev
        prev = current
        current = next
    }

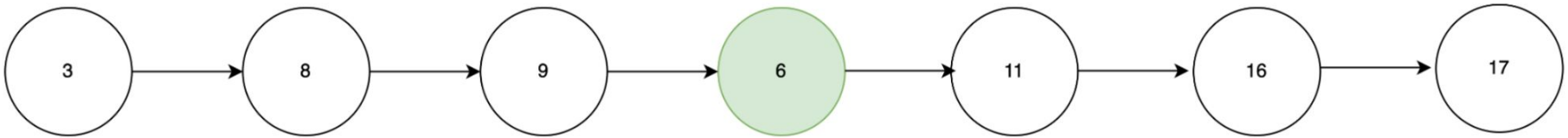
    head = prev
    return head
}
```

Найти середину списка

Дан связный список.

Необходимо найти середину списка.
Сделать это необходимо за O(n)
без дополнительных аллокаций

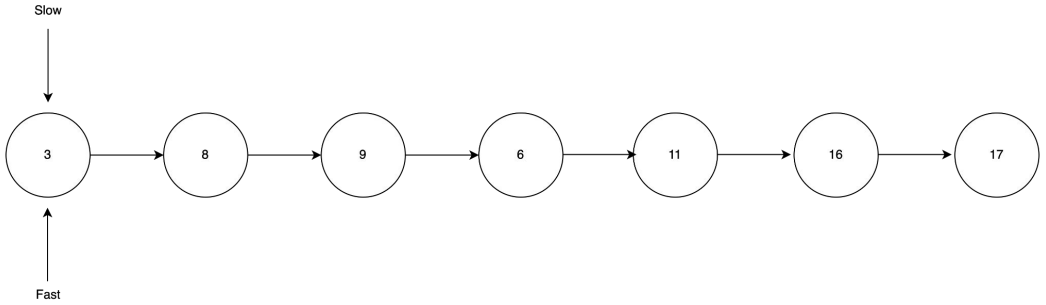
Середина связного списка

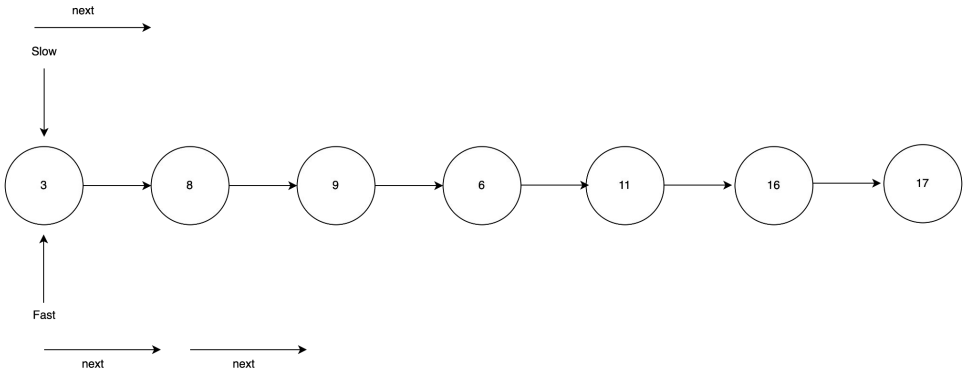


Середина связного списка

Применим два указателя.

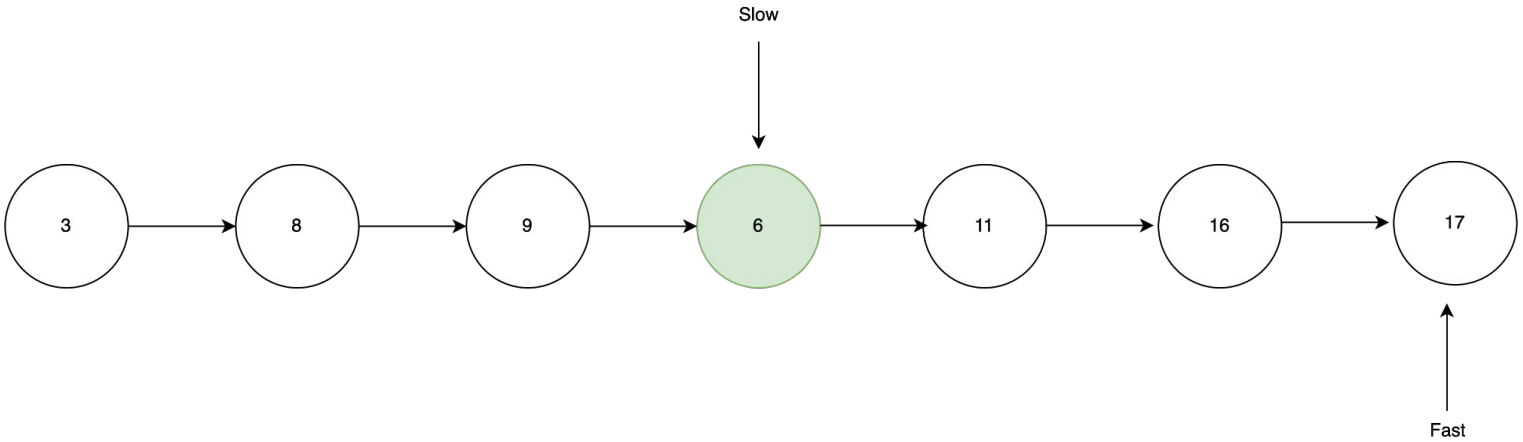
Один будет двигаться вперед на next, а другой на next.next





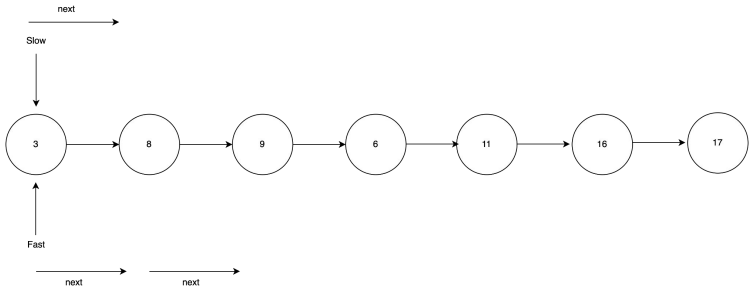
Середина связного списка

Когда быстрый указатель достигнет конца списка, указатель slow будет указывать на середину



Середина связного списка

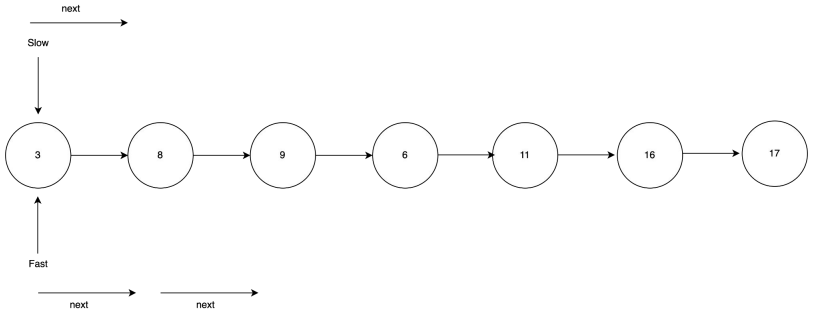
Когда быстрый указатель достигнет конца списка, указатель slow будет указывать на середину




```
function middleNode(head) {
    slow = fast = head
    // . . .?
    return slow
}
```

Середина связного списка

Когда быстрый указатель достигнет конца списка, указатель slow будет указывать на середину

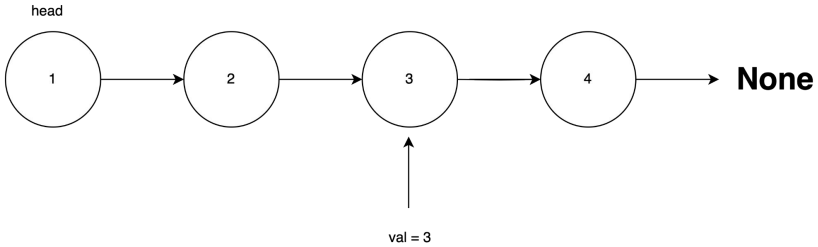


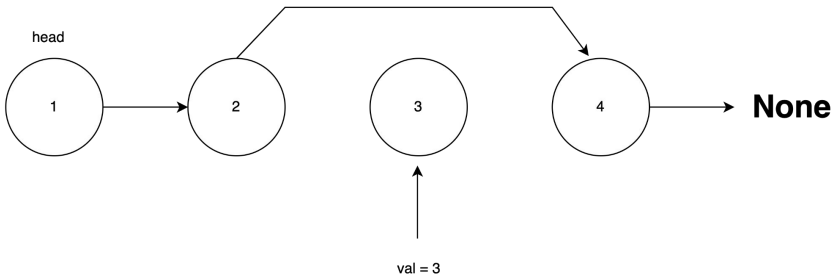

```
function middleNode(head) {
    slow = fast = head
    while fast != null and fast.next != null {
        slow = slow.next
        fast = fast.next.next
    }
    return slow
}
```

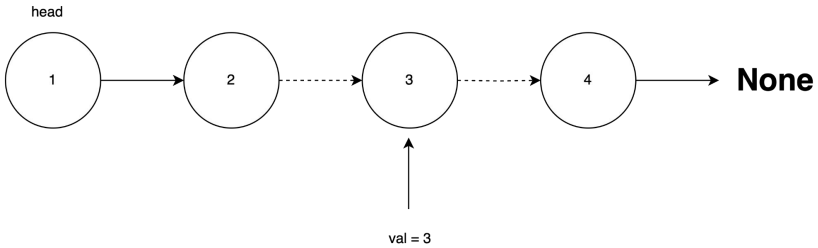
Удалить элемент из односвязного списка

Необходимо написать функцию, которая принимает на вход односвязный список и некоторое целое число val. Необходимо удалить узел из списка, значение которого равно val.

Удаление элемента из связного списка

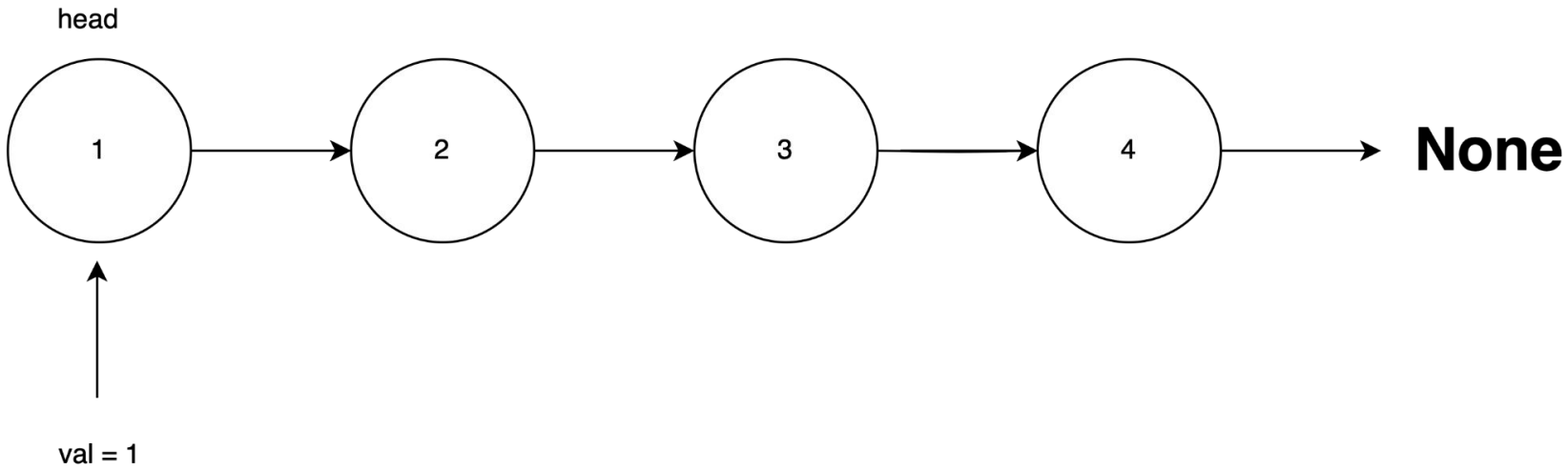






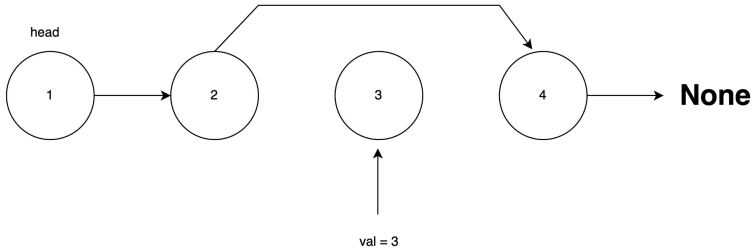
Удаление элемента из связного списка

А как быть если, мы хотим удалить первый элемент списка?



removeElement

Создаем переменную dummy, основная задача которой — быть заглушкой на случай если придется удалять первый элемент.




```
class ListNode {
    constructor(val) {
        this.val = val
        this.next = null
    }
}

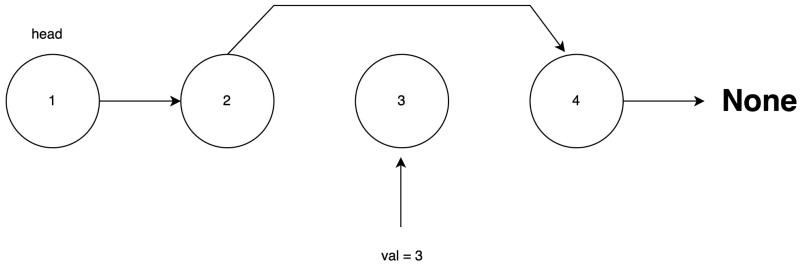
function removeElements(head, val) {
    dummy = new ListNode()
    dummy.next = head
    prev = dummy
    cur = head

    while (cur != null) {
    // . . . ?
    }

    return dummy.next
}
```

removeElement

Создаем переменную dummy, основная задача которой — быть заглушкой на случай если придется удалять первый элемент.




```
class ListNode {
    constructor(val) {
        this.val = val
        this.next = null
    }
}

function removeElements(head, val) {
    dummy = new ListNode()
    dummy.next = head
    prev = dummy
    cur = head

    while (cur != null) {
        if (cur.val == val) {
            prev.next = cur.next
        } else {
            prev = cur
        }
        cur = cur.next
    }

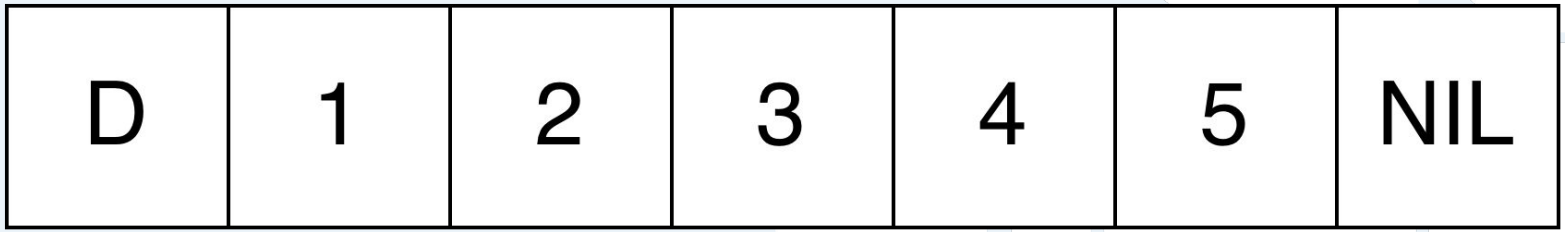
    return dummy.next
}
```

Удаление n-ого элемента с конца списка

Дан head односвязного списка и целое число n. Необходимо удалить n-й узел с конца и вернуть обновлённый список. Например, если вход - [1,2,3,4,5] и n = 2, результатом будет [1,2,3,5].

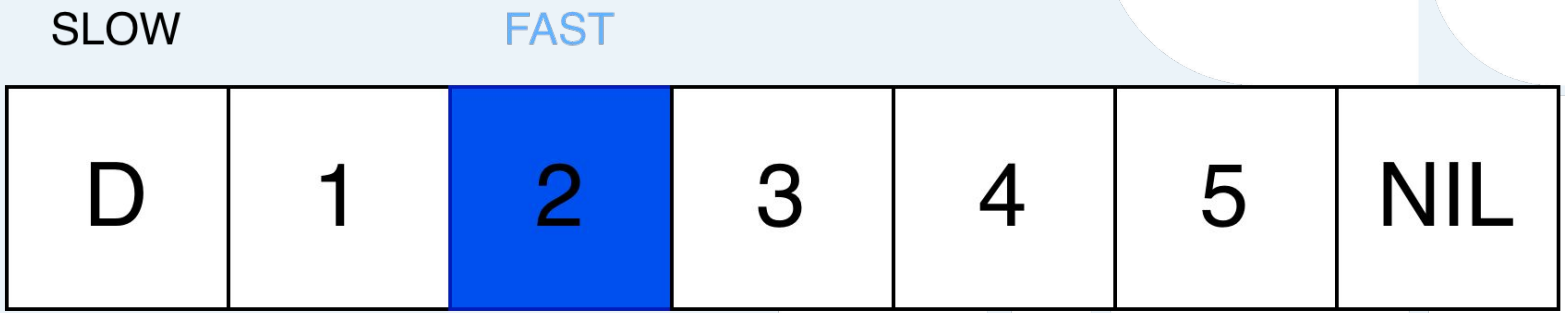
Удаление n-ого элемента с конца списка

Список: dummy -> 1 -> 2 -> 3 -> 4 -> 5 -> nil, n = 2
Нужно удалить 2-й узел с конца (узел со значением 4).
Идея: два указателя fast и slow, оба стартуют с dummy-узла.



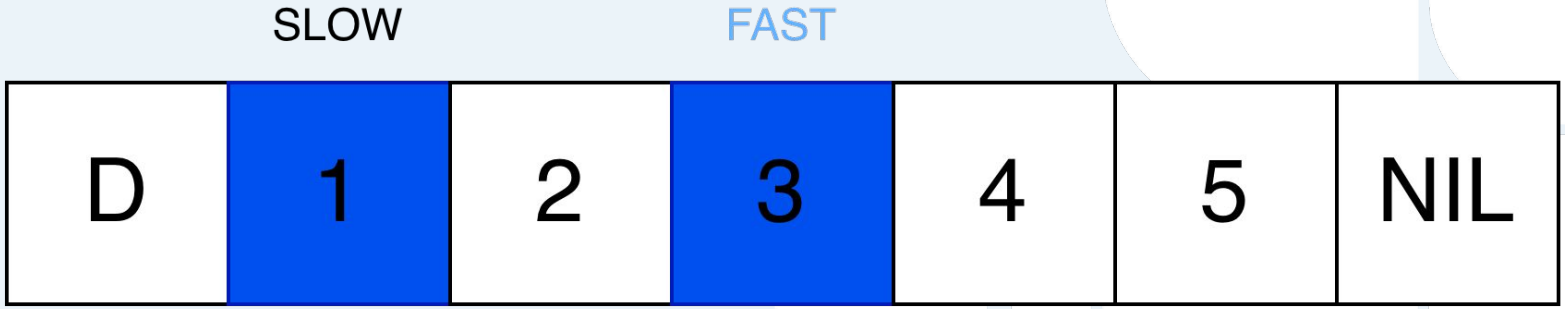
Удаление n-ого элемента с конца списка

Шаг 1: двигаем fast вперед на n=2 шага.
fast: dummy -> 1 -> 2 (сделано 2 шага)



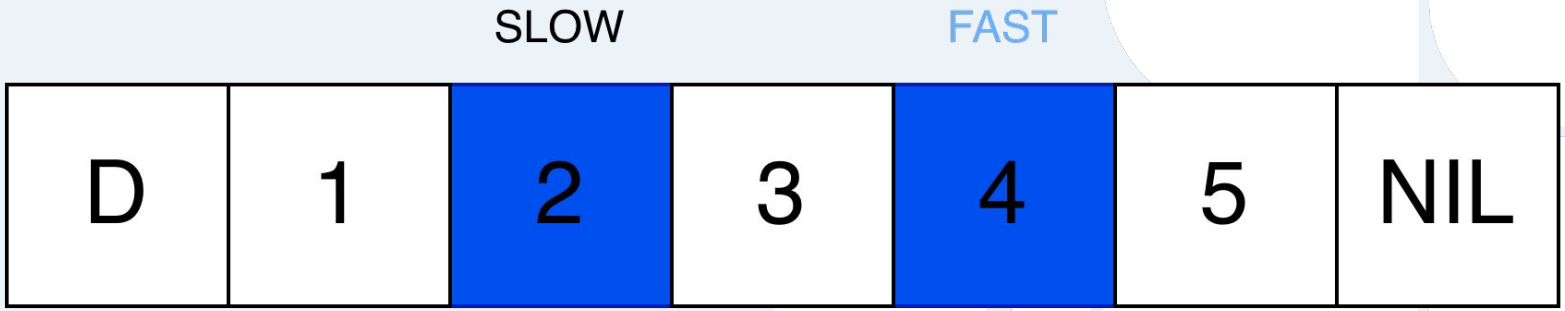
Удаление n-ого элемента с конца списка

Шаг 2: двигаем fast и slow одновременно, пока fast.Next != nil.
Итерация 1: fast -> 3, slow -> 1



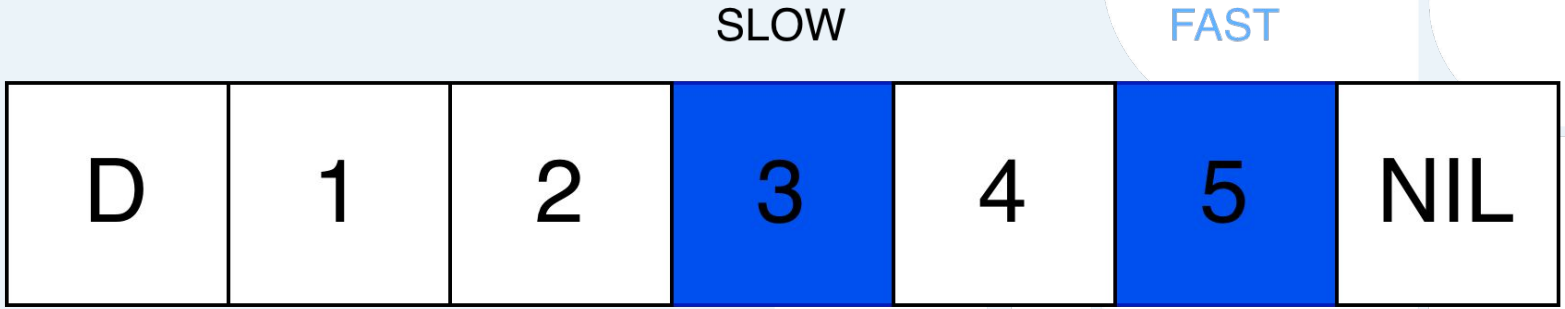
Удаление n-ого элемента с конца списка

Итерация 2: fast -> 4, slow -> 2



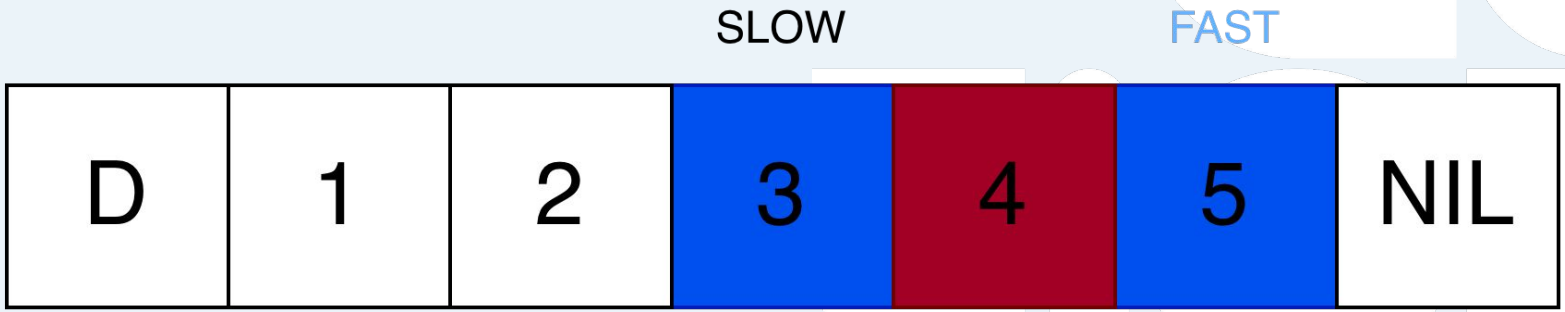
Удаление n-ого элемента с конца списка

Итерация 3: fast -> 5, slow -> 3
fast.Next == nil, останавливаемся.



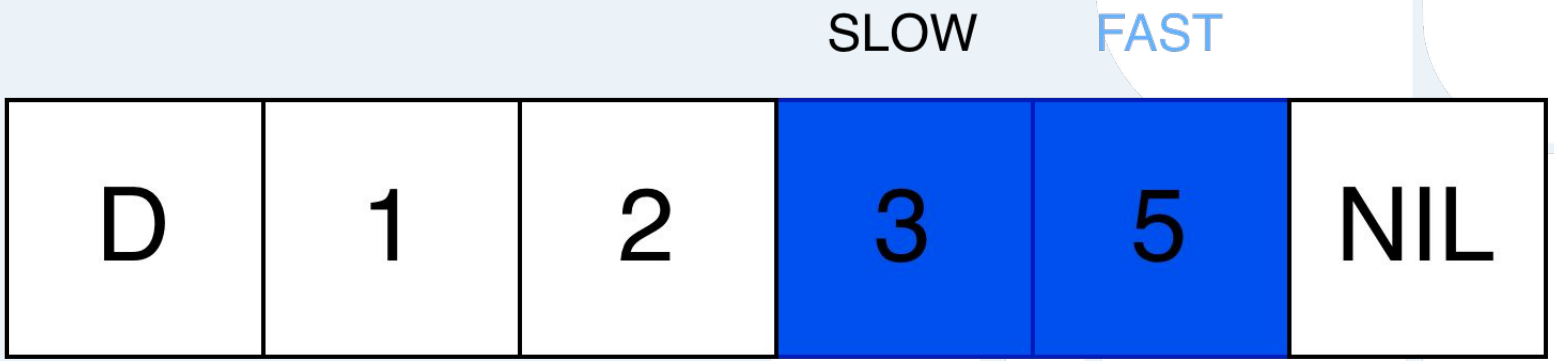
Удаление n-ого элемента с конца списка

slow стоит на узле 3 (предшественник удаляемого узла 4).
Выполняем: slow.Next = slow.Next.Next
Узел 4 исключён из списка!



Удаление n-ого элемента с конца списка

Результат: dummy -> 1 -> 2 -> 3 -> 5 -> nil
Возвращаем dummy.Next = head обновлённого списка.



```
function removeNthFromEnd(head, n) {
 dummy = new ListNode(0, head);
 dummy.next = head
 slow = dummy;
 fast = dummy;

 for (i = 0; i <= n; i++) {
   fast = fast.next;
 }

 while (fast != null) {
   slow = slow.next;
   fast = fast.next;
 }

 slow.next = slow.next.next;

 return dummy.next;
}
```

Является ли одна строка исходной для другой

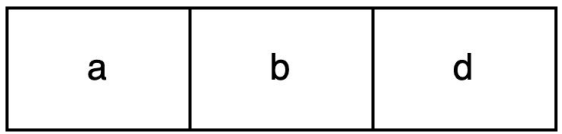
В исходную строку добавили некоторое количество символов.

Необходимо выявить, была ли строка a исходной для строки b.

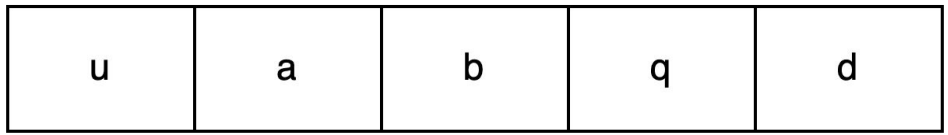
Является ли одна строка исходной для другой строки?

В исходную строку добавили некоторое количество символов.
Необходимо выявить, была ли строка a исходной для строки b.

строка A



строка B



Является ли одна строка исходной для другой строки?

- Инициализируем очередь
- Заполняем ее значениями из строки a

```
function isSubsequence(a, b) {
    q = Queue()
    for el in a {
        q.push(el)
    }
}
```

Является ли одна строка исходной для другой строки?

В цикле по строке b необходимо проверить вхождения элементов из строки a в строку b

```
function isSubsequence(a, b) {
    q = Queue()
    for el in a {
        q.push(el)
    }
    for el in b {
        …
    }
}
```

Является ли одна строка исходной для другой строки?

Если элемент, который следующий
в очереди, равен текущему элементу
в строке b, то мы его удаляем из очереди

```
function isSubsequence(a, b) {
    q = Queue()
    for el in a {
        q.push(el)
    }
    for el in b {
        if q.peek() == el {
            q.pop()
        }
    }
}
```

Является ли одна строка исходной для другой строки?

Если очередь в конце окажется пустой, значит строка a — исходная для строки b

```
function isSubsequence(a, b) {
    q = Queue()
    for el in a {
        q.push(el)
    }
    for el in b {
        if q.peek() == el {
            q.pop()
        }
    }
    return q.getSize() == 0
}
```

Является ли одна строка исходной для другой строки?

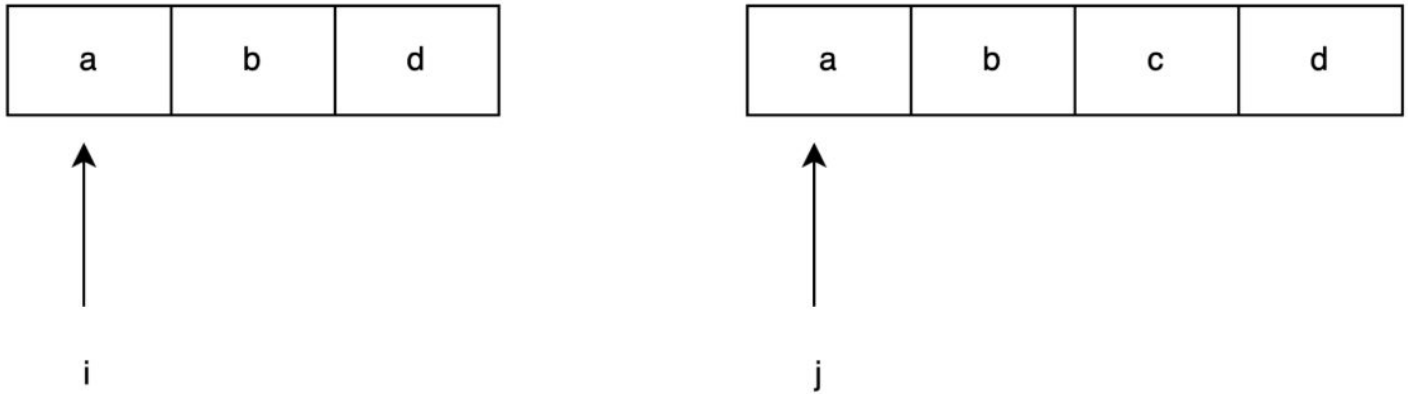
Сложность O(n)

Необходима дополнительная аллокация памяти O(n)

```
function isSubsequence(a, b) {
    q = Queue()
    for el in a {
        q.push(el)
    }
    for el in b {
        if q.peek() == el {
            q.pop()
        }
    }
    return q.getSize() == 0
}
```

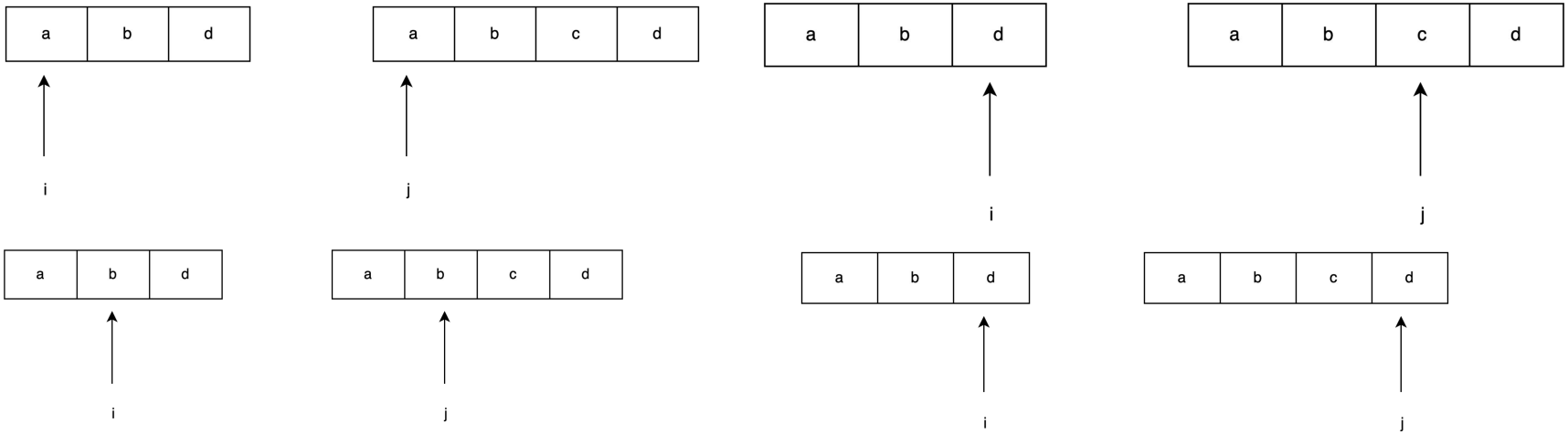
Метод двух указателей

Является ли одна строка исходной для другой строки?



Является ли одна строка исходной для другой строки?

В случае равенства элементов под индексами i и j мы двигаем оба указателя.
В противном случае только j.



Является ли одна строка исходной для другой строки?

- i будет идти по строке a
- j будет идти по строке b

```
function isSubsequence(a, b) {
    i,j = 0,0
}
```

Является ли одна строка исходной для другой строки?

Цикл продолжается пока не дойдем до конца одной из строк

```
function isSubsequence(a, b) {
    i,j = 0,0
    while i < len(a) and j < len(b){
      …
    }
}
```

Является ли одна строка исходной для другой строки?

Если элементы не равны, то двигаем только указатель у строки b

```
function isSubsequence(a, b) {
    i,j = 0,0
    while i < len(a) and j < len(b){
        if a[i] == b[j] {
            i++
        }
        j++
    }
}
```

Является ли одна строка исходной для другой строки?

Если мы дошли до конца строки a, значит a — исходная строка для b

```
function isSubsequence(a, b) {
    i,j = 0,0
    while i < len(a) and j < len(b){
        if a[i] == b[j] {
            i++
        }
        j++
    }
    return i == len(a)
}
```

Является ли одна строка исходной для другой строки?

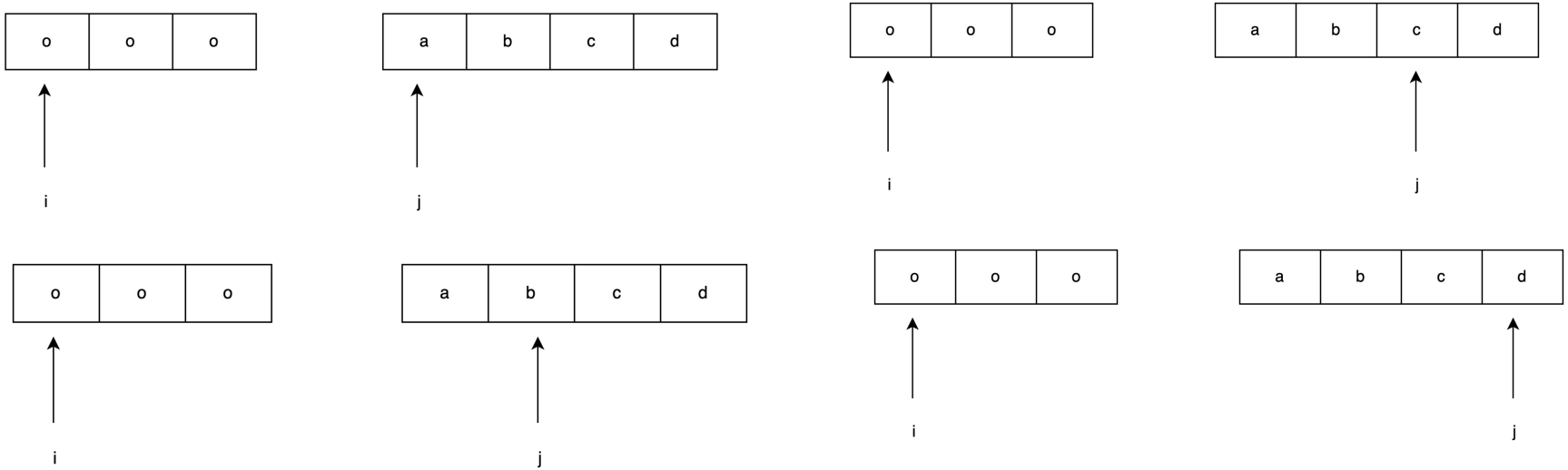
Сложность O(n)

Не требуется дополнительной памяти

```
function isSubsequence(a, b) {
    i,j = 0,0
    while i < len(a) and j < len(b){
        if a[i] == b[j] {
            i++
        }
        j++
    }
    return i == len(a)
}
```

Является ли одна строка исходной для другой строки?

Значение i останется равным нулю.



Является ли одна строка исходной для другой строки?

Наивная реализация O(n^2)

```
function isSubsequence(a, b) {
 i = 0;
 lenA = len(a);
 lenB = len(b);

 for (el1 = 0; el1 < lenA; el1++) {
   for (el2 = 0; el2 < lenB; el2++) {
     if (a[el1] == b[el2]) {
       i++;
       break; // Прерываем внутренний цикл
     }
   }
 }

 return i === lenA;
}
```

Является ли слово палиндромом?

Напишите функцию, которая принимает на вход строку и возвращает true, если она является палиндромом*.
В противном случае верните false.

*слово или текст, одинаково читающиеся в обоих направлениях.

Является ли слово палиндромом?

- Инициализируем стек
- Заполняем его значениями из входной строки

```
function isPalindrome(s) {
    stack = Stack()
    for char in s {
        stack.push(char)
    }
}
```

Является ли слово палиндромом?

- В цикле проходим по строке
- На каждой итерации берем букву из вершины стека
- Если она не равна текущему символу из входной строки, возвращаем false

```
function isPalindrome(s) {
    stack = Stack()
    for char in s {
        stack.push(char)
    }
    for char in s {
        if char != stack.pop() {
            return false
        }
    }
}
```

Является ли слово палиндромом?

Если по ходу цикла все символы из строки совпали с символами из стека, то значит слово является палиндромом

```
function isPalindrome(s) {
    stack = Stack()
    for char in s {
        stack.push(char)
    }
    for char in s {
        if char != stack.pop() {
            return false
        }
    }
    return true
}
```

Является ли слово палиндромом?

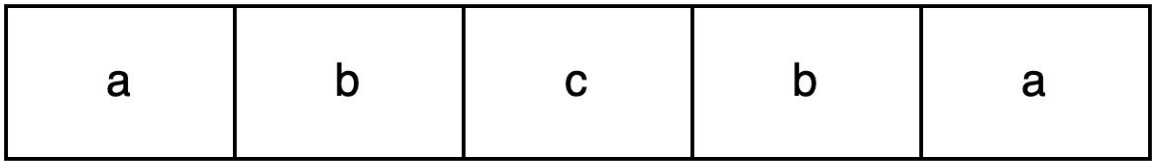
Сложность O(n)

Необходима дополнительная аллокация памяти O(n)

```
function isPalindrome(s) {
    stack = Stack()
    for char in s {
        stack.push(char)
    }
    for char in s {
        if char != stack.pop() {
            return false
        }
    }
    return true
}
```

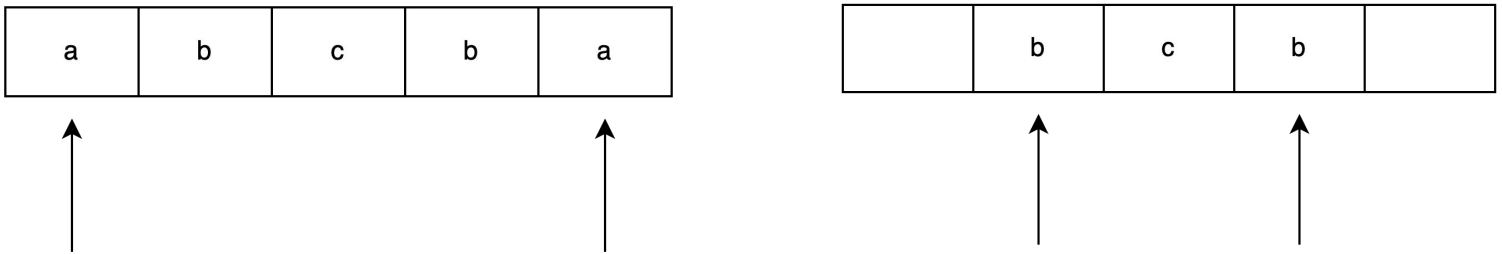
Является ли слово палиндромом?

Используя deque

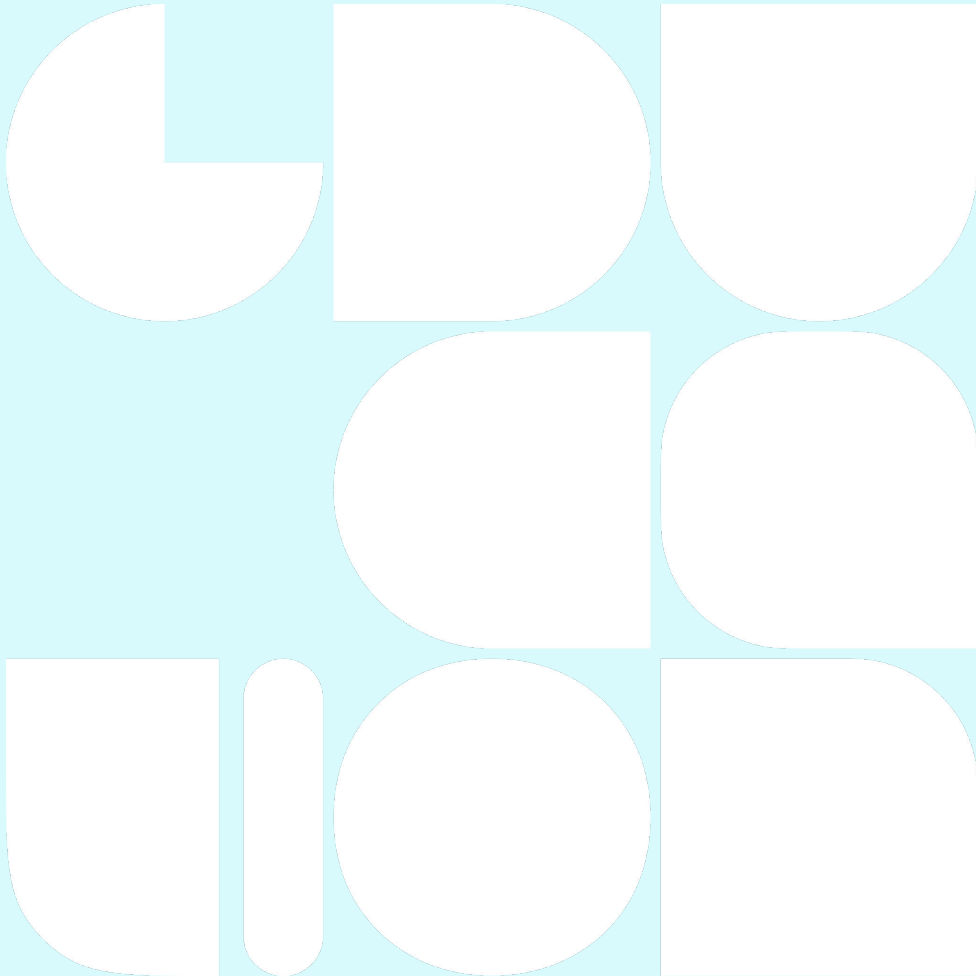


Является ли слово палиндромом?

Используя deque



Метод двух указателей



Является ли слово палиндромом?

- Инициализируем left и right

```
function isPalindrome(s) {
 left = 0;
 right = len(s) - 1;

 return true;
}
```

Является ли слово палиндромом?

- Цикл продолжается, пока left и right не равны
- В случае, если элементы под этими индексами не равны, то возвращаем false

```
function isPalindrome(s) {
 left = 0;
 right = len(s) - 1;

 while (left < right) {
   if (s[left] != s[right]) {
     return false;
   }
   …
 }

 return true;
}
```

Является ли слово палиндромом?

На каждой итерации двигаем left вправо и right влево

```
function isPalindrome(s) {
 left = 0;
 right = len(s) - 1;

 while (left < right) {
   if (s[left] !== s[right]) {
     return false;
   }
   left++;
   right--;
 }

 return true;
}
```

Является ли слово палиндромом?

Сложность O(n)

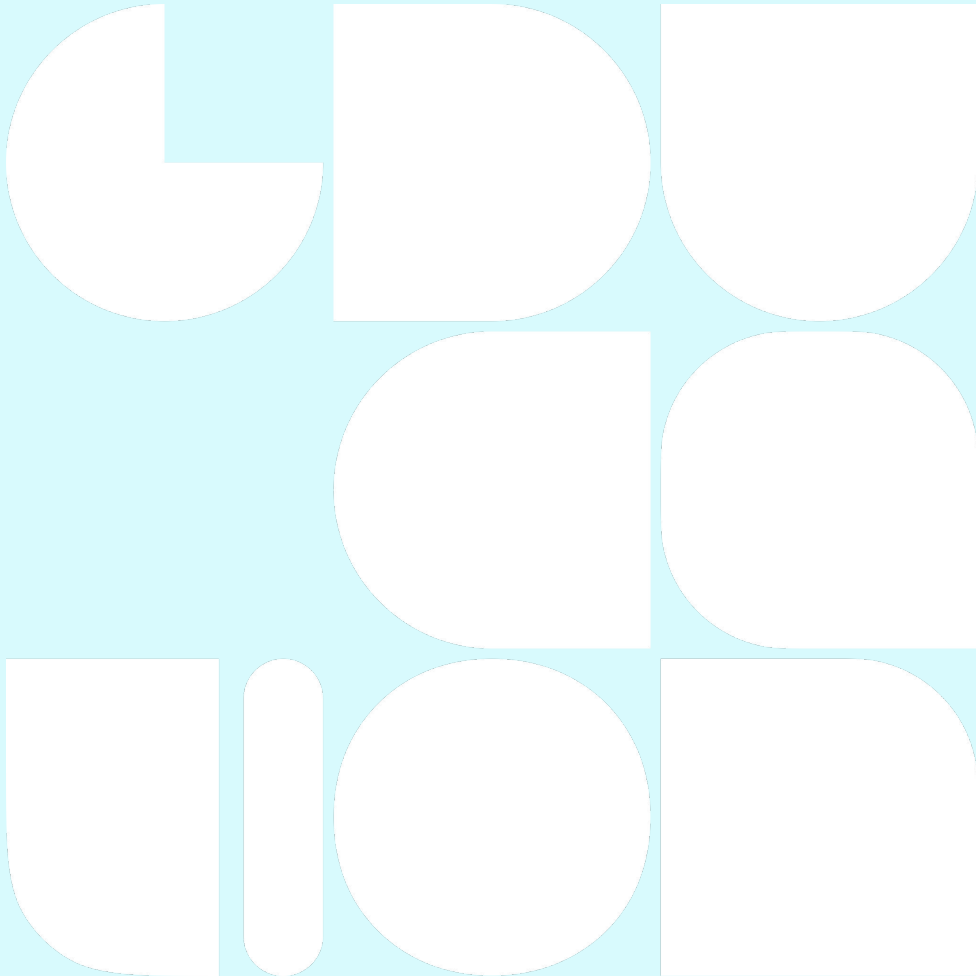
Нет необходимости в дополнительной памяти

```
function isPalindrome(s) {
 left = 0;
 right = len(s) - 1;

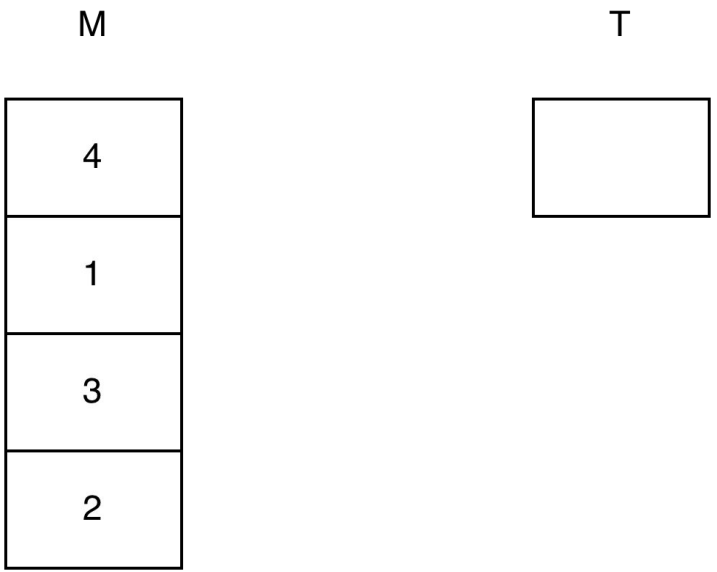
 while (left < right) {
   if (s[left] !== s[right]) {
     return false;
   }
   left++;
   right--;
 }

 return true;
}
```

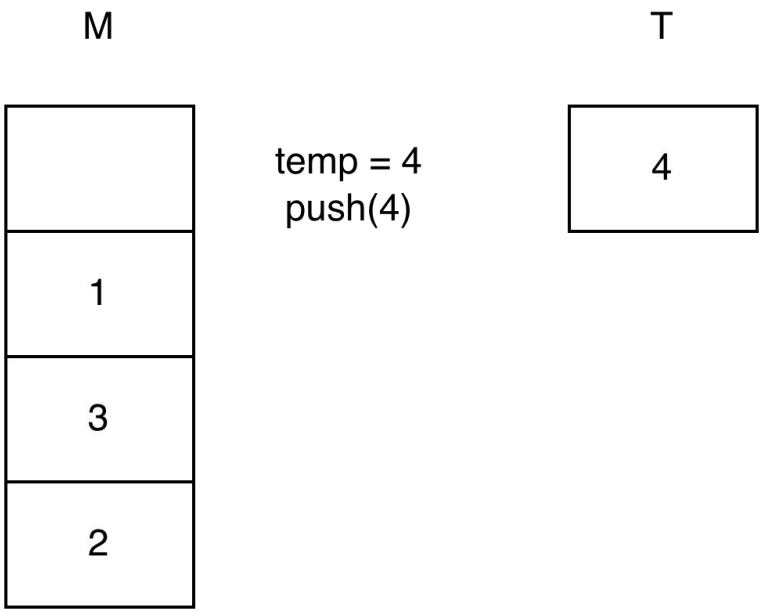
Сортировка стека через дополнительный стек



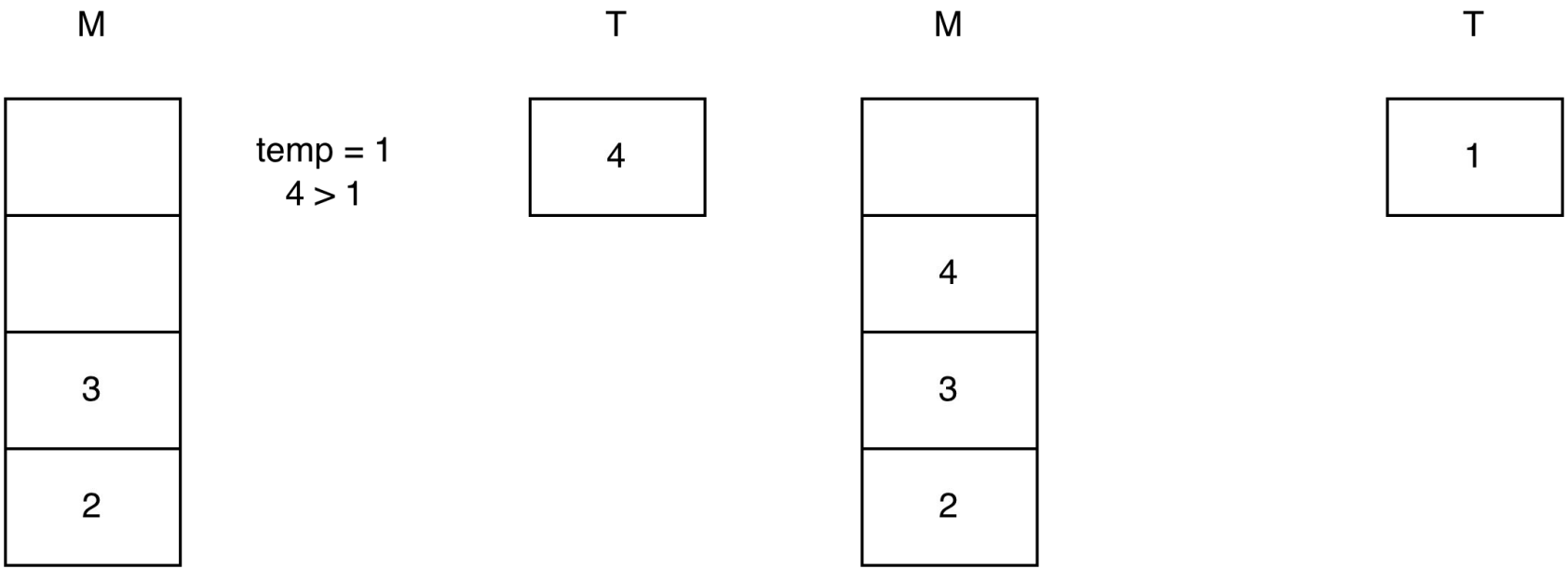
Сортировка массива через дополнительный стек



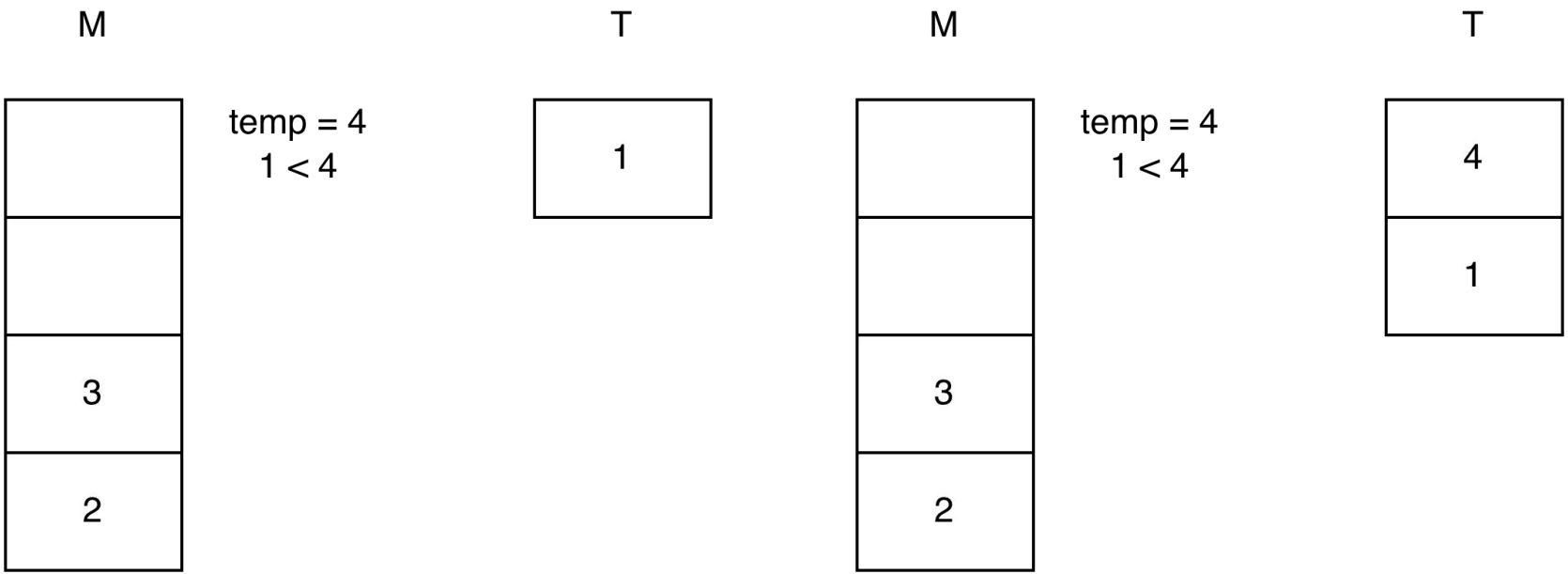
Сортировка массива через дополнительный стек



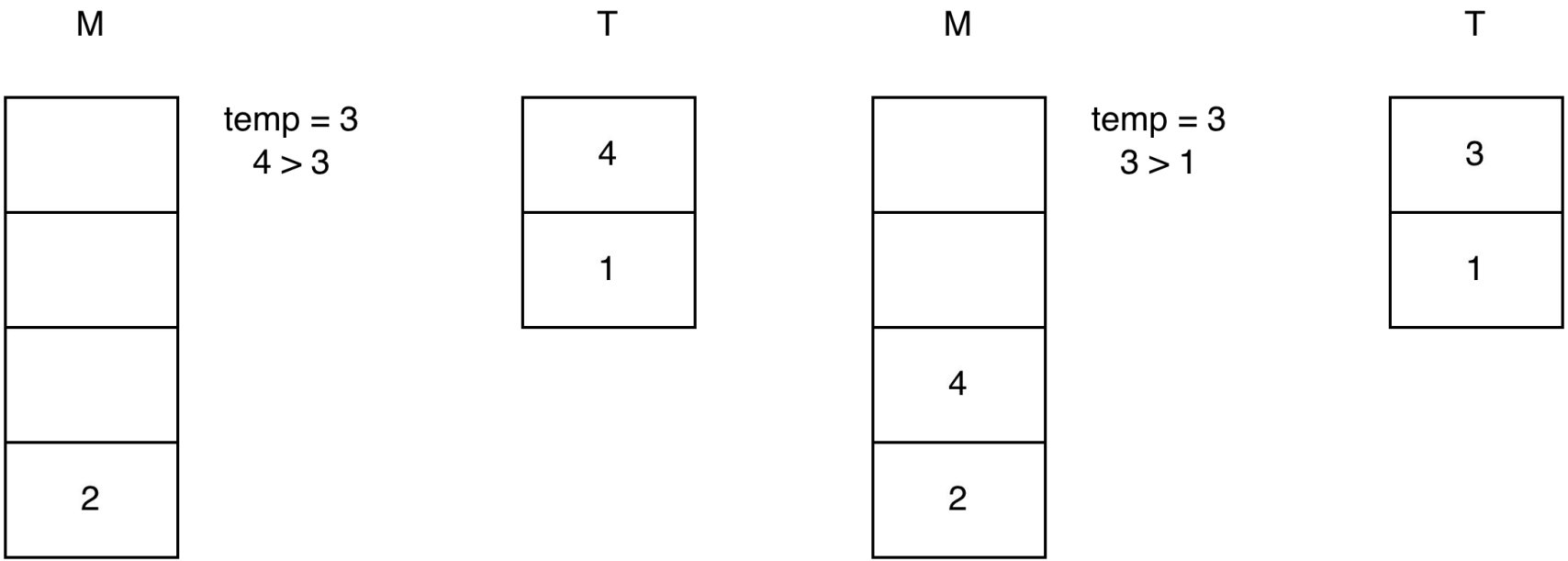
Сортировка массива через дополнительный стек



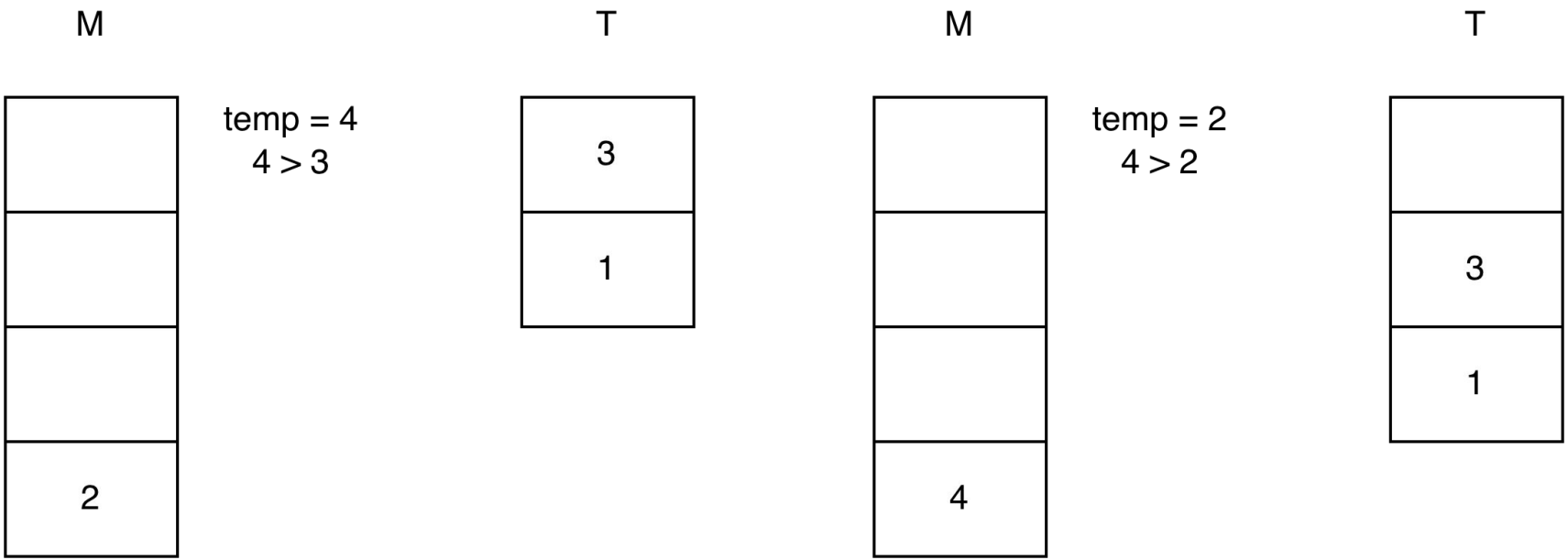
Сортировка массива через дополнительный стек



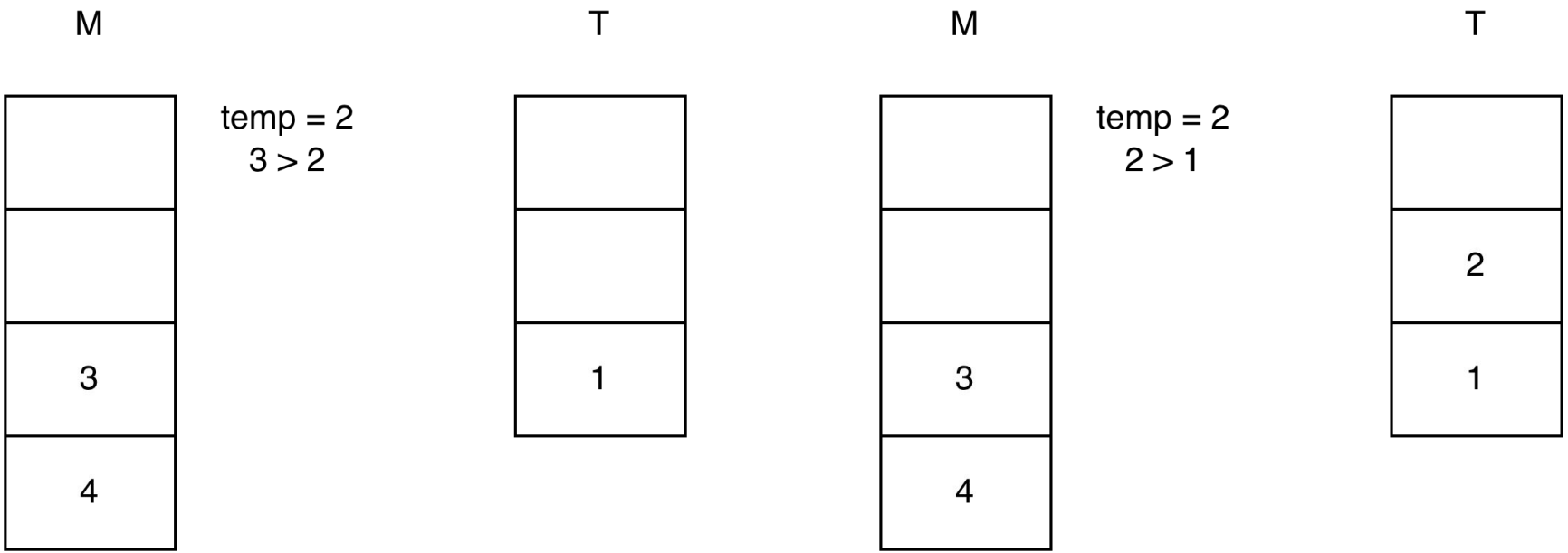
Сортировка массива через дополнительный стек



Сортировка массива через дополнительный стек



Сортировка массива через дополнительный стек



Сортировка массива через дополнительный стек

```
function sortArrayUsingStack(input) {
    stack = []; // дополнительный стек

    while (input.length > 0) {
        current = input[input. length - 1];

        // Перекладываем из доп. стека обратно в input все элементы,
        // которые больше current
        while (stack.length > 0 && stack[stack. length - 1] > current) {
            input. append(stack.pop());
        }

        stack.push(current);
    }

    return stack;
}
```

Слияние двух отсортированных списков

Написать функцию, которая принимает на вход два отсортированных односвязных списка и объединяет их в один отсортированный список. При этом затраты по памяти должны быть O(1)

Входные данные: list1 = [3, 6, 8], list2 = [4, 7, 9, 11]
Выходные: [3,4,6,7,8,9,11]

Спасибо за внимание!

education